

HW1

```
aa <- read.delim("SRBCT_train.txt") # 讀檔  
bb <- as.matrix(aa[,-c(1,2)]) # 扣掉第 1、2 col，因為是  
character  
dim(bb) # 觀察維度
```

Output:

```
[1] 2308 63
```

題目一：Get mean & sd value for each gene cross samples

Get mean value for each gene cross samples

```
# 求 means  
# 方法一 -> 使用 apply  
mean_gene_cross_samples1 <- apply(bb, 1, mean)  
mean_gene_cross_samples1  
  
# 方法二 -> 使用內建函式 rowMeans  
mean_gene_cross_samples2 <- rowMeans(bb)
```


審視多種向量比較方法及其應用場景。

在 R 裡驗證兩向量是否相同，會依「相同」的定義不同而有幾種做法。以下由嚴格到寬鬆說明。

1. `identical()` —— 最嚴格

檢查型別、長度、順序、屬性全部完全一致，回傳單一的 `TRUE` / `FALSE`。這是判斷「完全相同」最可靠的方式。

```
r
x <- c(1, 2, 3)
y <- c(1, 2, 3)
identical(x, y) # TRUE
```

要注意型別差異它會抓出來：

```
r
identical(c(1, 2, 3), c(1L, 2L, 3L)) # FALSE (double vs integer)
```

2. `all(x == y)` —— 逐元素比較

先用 `==` 逐一比對，再用 `all()` 確認是否全部為真。比 `identical()` 寬鬆，不管型別細節。

```
r
all(c(1, 2, 3) == c(1L, 2L, 3L)) # TRUE
```

但這個寫法有兩個陷阱要小心：

- 長度不同時會觸發回收 (recycling)，可能給出誤導的結果，甚至只是警告而非錯誤。保險起見可以先比長度：`length(x) == length(y) && all(x == y)`。
- 含有 `NA` 時，`==` 會回傳 `NA`，導致 `all()` 也變成 `NA`。可以加 `all(x == y, na.rm = TRUE)`，但更穩健的還是用 `identical()`。

3. `all.equal()` —— 容許浮點誤差

比較數值時，因為浮點運算的精度問題，`==` 有時會失準。`all.equal()` 允許微小的容差：

```
r
all(0.1 + 0.2 == 0.3) # FALSE ! 浮點誤差
isTRUE(all.equal(0.1 + 0.2, 0.3)) # TRUE
```

注意 `all.equal()` 在不相等時回傳的是「差異的描述文字」而非 `FALSE`，所以通常要包一層 `isTRUE()` 來取得乾淨的邏輯值。

題目二：Get mean & sd value for each sample cross genes

Get mean value for each sample cross genes

```
# 題目二：Get mean & sd value for each sample cross genes

# 求 means
# 方法一
mean_sample_cross_genes1 <- apply(bb, 2, mean)
# 方法二
mean_sample_cross_genes2 <- colMeans(bb)
```

Ouput:

```
[1] 0.8999231 0.9107066 0.9121858 0.9216109 0.9393284
0.9246767 0.9181976 0.9211571 0.9080536 0.9379128 0.9000853
0.9039657 0.9176309
[14] 0.8991331 0.8911080 0.8915395 0.9212825 0.9070417
0.9039065 0.8974366 0.8833335 0.8983932 0.8853999 0.8999410
0.8912325 0.8921722
[27] 0.8891327 0.8815866 0.8788212 0.8820618 0.8865663
0.8988042 0.9210532 0.9083504 0.9061526 0.8964495 0.9260049
0.9098169 0.9065384
[40] 0.9023300 0.8988161 0.8996560 0.8955150 0.9109578
0.9330225 0.9104899 0.9240139 0.9156453 0.9120389 0.9096284
0.9137515 0.9013536
[53] 0.9162928 0.9240076 0.9189305 0.9310080 0.9385528
0.9221431 0.9259349 0.9295506 0.9279080 0.9167595 0.8906594
```

驗證結果是否相同

```
all.equal(mean_sample_cross_genes1, mean_sample_cross_genes2)
```

Ouput:

```
[1] TRUE
```

Get sd value for each sample cross genes

```
# 求 sd
sd_sample_cross_genes <- apply(bb, 2, sd)
sd_sample_cross_genes
```

Output:

```
EWS.T1 EWS.T2 EWS.T3 EWS.T4 EWS.T6 EWS.T7 EWS.T9 EWS.T11 EWS.T12 EWS.T13 EWS.T14 EWS.T15 EWS.T19
0.9023934 0.8952745 0.9059050 1.1925464 0.8663332 0.9343484 0.8483575 1.0391300 1.1529887 1.3418554 1.1775772 1.2658881 1.2677206
EWS.C8 EWS.C3 EWS.C2 EWS.C4 EWS.C6 EWS.C9 EWS.C7 EWS.C1 EWS.C11 EWS.C10 BL.C5 BL.C6 BL.C7
1.1612246 0.9871340 0.9954100 0.7455796 0.8318380 0.9232795 1.0691206 1.0979321 0.9725914 1.1268339 0.9649714 0.9442416 1.0255234
BL.C8 BL.C1 BL.C2 BL.C3 BL.C4 NB.C1 NB.C2 NB.C3 NB.C6 NB.C12 NB.C7 NB.C4 NB.C5
1.0811283 1.0854579 1.1259233 0.9222812 0.9586906 0.9186860 0.8783569 1.1563298 1.0061901 1.0522149 0.9551821 0.8653178 0.9557501
NB.C10 NB.C11 NB.C9 NB.C8 RMS.C4 RMS.C3 RMS.C9 RMS.C2 RMS.C5 RMS.C6 RMS.C7 RMS.C8 RMS.C10
0.7951259 0.9779185 0.9523493 1.1118753 1.2229604 0.8485316 1.4133762 1.2185764 0.9787163 0.8492464 1.0370480 0.8156487 1.1493356
RMS.C11 RMS.T1 RMS.T4 RMS.T2 RMS.T6 RMS.T7 RMS.T8 RMS.T5 RMS.T3 RMS.T10 RMS.T11
0.7998934 0.9975282 0.9611903 0.8886484 1.1732700 1.7319878 1.0980215 1.0859599 1.0401830 1.1738220 1.6555917
```

Select top 30% genes with larger sd values

```
# AI
n_top30_cols <- ceiling(nrow(bb) * 0.3)
top30_genes_sd <- order(sd_gene_cross_samples, decreasing =
TRUE)[1:n_top30_cols]
top30_genes_sd
```

Output:

```
[1] 509 1834 187 1781 1750 1774 62 1572 1547 672 1897 735 1574 1975 551 430 7 1517 146 60 246 11 364 329 292 469 520 151 148 545
[31] 22 523 334 1544 831 1372 540 24 1955 1645 996 554 141 360 276 49 992 1708 13 1065 6 1954 68 77 1345 1389 61 1915 331 326
[61] 26 544 294 330 572 566 991 1826 689 2099 349 1786 842 1653 1600 25 1093 1907 347 1319 254 2223 202 47 1573 761 1079 261 235 2046
[91] 302 1625 198 1448 51 1977 1070 1546 1621 83 886 2147 272 714 563 1044 826 951 448 2115 851 1771 35 937 1083 1540 1831 1851 1614 266
[121] 1594 536 541 1371 156 644 468 727 144 1072 1765 1927 1074 1478 475 1412 180 1932 1630 1947 1894 218 1299 1739 88 336 251 1113 124 1820
[151] 2290 101 252 1175 1222 355 1890 973 742 1150 1484 423 801 1626 874 901 57 129 872 855 910 573 1607 1243 1227 669 1682 781 107 470
[181] 55 912 593 811 524 1855 1084 1582 2135 1280 42 1615 298 1109 1871 800 84 220 604 547 1295 665 1648 1715 1105 538 1767 839 1256 1076
[211] 2230 2213 1339 1055 539 130 402 1031 1327 1277 1023 911 345 577 2077 1128 36 1021 200 1489 1841 1937 64 58 1353 48 2240 1965 132 1916
[241] 286 1602 1632 287 1601 241 1721 1810 1167 1483 94 1618 2292 1223 112 189 185 471 91 1529 1543 1185 889 1066 353 2273 1772 1090 368 1358
[271] 1497 721 866 1524 214 373 760 822 18 1114 1857 164 1603 905 606 1535 462 169 317 581 567 929 647 176 2277 983 297 1418 596 133
[301] 1169 232 2022 1135 1151 2117 205 1057 1397 758 289 699 458 869 1300 1426 2050 594 1075 1666 971 1961 230 1945 215 1843 844 281 1053 1872
[331] 1494 709 1980 1738 134 1382 438 1814 346 649 1048 979 1183 574 617 1166 883 925 1769 1349 2029 59 1 19 127 719 1900 584 1940 657
[361] 1417 2228 1956 2020 85 1759 33 1307 922 1542 1780 1219 2234 576 348 183 560 1616 99 1803 1654 76 295 34 775 2263 739 1162 720 1885
[391] 2199 1311 1960 974 976 1271 162 752 924 859 1736 501 1764 1067 1973 1748 1690 236 2218 460 904 1116 631 1683 118 970 1795 140 1024 1726
[421] 46 671 522 308 434 838 472 963 682 488 975 14 1730 1812 1310 1229 1752 510 1022 1799 1783 352 1588 153 412 528 741 2220 1972 1623
[451] 724 836 1620 1029 65 1525 619 1596 1568 762 2239 1177 588 1884 1651 1213 776 1858 877 1452 1612 1318 1604 1913 1158 159 1792 285 420 1141
[481] 1883 371 30 1309 1362 316 614 1718 2256 178 342 518 748 1032 1197 751 1901 455 1863 1986 1874 1391 1650 736 1052 27 1837 28 998 161
[511] 1692 1071 1608 2157 1272 1304 1207 21 1282 260 1414 899 1017 2260 87 256 1519 406 184 941 1813 1611 1821 296 422 2148 915 1548 341 1555
[541] 1541 1298 1144 1208 1981 41 1297 1724 1785 1822 846 247 978 1580 1237 945 1139 119 1447 756 53 989 943 126 10 1042 949 278 2171 873
[571] 301 802 549 207 370 634 603 1468 273 250 1725 638 980 1847 1886 473 1533 2303 1924 597 1069 149 1062 774 2221 320 571 1473 532 1094
[601] 1203 2270 1305 857 819 1374 1154 1892 2007 1678 476 1836 152 1402 1844 1441 796 56 1950 136 3 192 625 1015 1260 2139 20 2249 936 115
[631] 933 633 231 2048 952 1922 1025 2113 824 135 1235 2093 2102 1020 730 381 1028 779 2231 1340 1082 957 1281 206 1959 1258 1122 362 1528 1714
[661] 1224 380 1099 788 546 1125 9 1323 1689 667 1200 1416 1550 507 212 1966 1003 729 314 1379 1342 137 607 615 930 2182 1039 74 1644 343
[691] 335 1078 1978
```

ceiling是什麼函式

下午2:01

`ceiling()` 是 R 的無條件進位函式，把數值往上取到最接近的整數。

```
r
ceiling(4.1) # 5
ceiling(4.9) # 5
ceiling(4.0) # 4
ceiling(-4.1) # -4 (往上取，所以是 -4 而非 -5)
```

負數要特別留意：「往上」是指往數線的正方向，所以 `-4.1` 進位後是 `-4`。

一組相關的取整函式對照：

- `ceiling(x)`：往上進位 → `ceiling(4.1)` 得 `5`
- `floor(x)`：往下捨去 → `floor(4.9)` 得 `4`
- `round(x)`：四捨五入（實際上是「四捨六入五成雙」的銀行家捨入）→ `round(2.5)` 得 `2`
- `trunc(x)`：直接去掉小數部分 → `trunc(-4.9)` 得 `-4`

一個常見用途是計算「需要多少組」，例如 23 個東西每組 5 個要幾組：

```
r
ceiling(23 / 5) # 5
```